AIREV-Agent-0.8B: Agentic AI on the Edge

Training Sub-Billion Parameter Models for Production Tool Calling
via Group Relative Policy Optimization with Progressive Curriculum

Muhammed Khalid
AIREV, On-Demand.io
mk@airev.ae @MeetsKhalid

March 2026

Abstract

We present **AIREV-Agent-0.8B**, a 752-million parameter language model fine-tuned for agentic tool calling that **outperforms the 11× larger Qwen3.5-9B base model on 7 out of 19 BFCL V4 categories**, including perfect scores on parallel function calling (10/10) and near-perfect on simple Python (10/10), Java (9/10), and multi-function selection (9/10). Built on Qwen3.5-0.8B's Gated Delta Network (GDN) architecture—262K context with hybrid linear/full attention—our model demonstrates that sub-billion parameter models can match or exceed models an order of magnitude larger on structured tool calling.

Our approach introduces four key innovations: (1) **Supervised fine-tuning on 50,000 Claude Opus 4.6-generated BFCL-format samples** with chain-of-thought reasoning; (2) **AutoResearch**—a Karpathy-style self-improving research loop scaled to 4 GPUs in parallel, running 240+ experiments to discover optimal GRPO configurations; (3) **GRPO on 43,097 clean training samples** combining real BFCL ground truth with diverse synthetic data; and (4) **Targeted SFT recovery** for multi-turn, memory, and web search categories using BFCL's actual function schemas.

We document critical findings: (a) AutoResearch discovered that optimal hyperparameters *change dramatically* when evaluated against 5 vs 19 BFCL categories—`lr=8e-6` was optimal for easy categories but `lr=2e-6` was optimal for the full distribution; (b) GRPO training on function calling *destroys* capabilities the base model already possessed—multi-turn accuracy dropped from 100% to 0% after GRPO; (c) 22% of Claude-generated irrelevance ground truth was incorrect, actively degrading model performance. All results reported are from internal evaluation; official BFCL V4 leaderboard scores are pending verification.

Model: `https://huggingface.co/airev-ai/AIREV-Agent-0.8B`

Model: `https://huggingface.co/airev-ai/AIREV-Agent-0.8B`

Code: `https://github.com/mk42-ai/AIREV-Agent-0.8B`

1 Introduction

The deployment of agentic AI systems—language models capable of autonomously selecting and invoking external tools—has been dominated by large-scale models with tens to hundreds of billions of parameters [Schick et al., 2023, Patil et al., 2023, Qin et al., 2024]. While these models achieve impressive tool-calling accuracy, their computational requirements preclude deployment on edge devices, mobile platforms, and resource-constrained environments where agentic AI could deliver the greatest practical value.

This paper addresses a fundamental question: *Can a sub-billion parameter model learn sophisticated tool-calling behavior—including multi-step dependency chains, parallel execution plans, and appropriate tool abstention—at accuracy levels competitive with frontier models?*

We answer affirmatively. Our model, **AIREV-Agent-0.8B**, achieves perfect 10/10 on `parallel` function calling, 10/10 on `simple_python`, 9/10 on `multiple`, and outperforms the 9B Qwen3.5 base model on 7 of 19 BFCL V4 categories—despite having only 0.8B parameters.

This work was motivated by production requirements at **On-Demand.io**[1], an agentic AI platform where user requests are routed through a plugin orchestration layer. The platform requires models that can:

1. Parse available tool schemas and select the correct tool(s) for a user request
2. Generate precise API parameters with correct types and values
3. Model dependencies between sequential tool calls
4. Identify when *no* available tool is relevant (irrelevance detection)
5. Execute on edge hardware with sub-100ms per-token latency

[1] `https://www.on-demand.io`

Our contributions are:

- A 0.8B model that outperforms the 11× larger Qwen3.5-9B on 7/19 BFCL categories, including +40% on Java and +34% on JavaScript function calling
- **AutoResearch for GRPO**: Karpathy-style automated research scaled to 4 parallel GPUs with eval-based ratcheting against 19 BFCL categories, discovering that optimal hyperparameters depend on the evaluation distribution (lr=8e-6 for easy categories, lr=2e-6 for the full benchmark)
- Discovery that **GRPO destroys pre-existing capabilities**: the base model scored 100% on multi-turn missing parameter detection, but GRPO training reduced this to 0% while improving function calling—a catastrophic forgetting trade-off not previously documented for GRPO
- Empirical evidence that Claude-generated ground truth contains 22% systematic errors on irrelevance detection, and that training on contaminated data actively degrades model performance

2 Related Work

Function Calling Benchmarks. The Berkeley Function Calling Leaderboard (BFCL) [Patil et al., 2024] established the standard evaluation framework for tool-calling capabilities, introducing Abstract Syntax Tree (AST) matching for robust evaluation across programming languages. BFCL v2 extended this with enterprise functions, and subsequent versions added multi-turn and agentic evaluation. Our evaluation uses the full BFCL V4 test suite (20 categories, 4,696 test samples), including multi-turn, memory, and web search categories.

Tool-Augmented Language Models. Toolformer [Schick et al., 2023] demonstrated self-supervised tool learning, while Gorilla [Patil et al., 2023] showed that fine-tuning on API documentation could match retrieval-augmented approaches. ToolLLM [Qin et al., 2024] introduced multi-tool reasoning chains. These works focus on large models (7B+); we extend this line to sub-1B models.

Reinforcement Learning for LLMs. GRPO [Shao et al., 2024] eliminates the need for a critic model by computing advantages relative to a group of sampled outputs, making RL feasible with limited GPU memory. DeepSeek-R1 [Guo et al., 2025] demonstrated that GRPO with verifiable rewards can teach structured reasoning. Our progressive curriculum approach builds on this by introducing reward components incrementally, analogous to curriculum learning [Bengio et al., 2009].

Efficient Architectures. Gated Delta Networks [Yang et al., 2024] combine linear attention (for efficient long-context processing) with sparse full attention layers, achieving state-of-the-art efficiency on long-sequence tasks. Qwen3.5 [Qwen Team, 2025] adopts this architecture, offering 262K context length in a 0.8B parameter model—an ideal foundation for edge deployment.

Small Model Tool Calling. Recent work on distilling tool-calling capabilities into small models includes Octopus [Chen et al., 2024], which fine-tuned 2B models for on-device function calling, and xLAM [Zhang et al., 2024], which explored function-calling-specific training at various scales. Our work pushes the boundary further to 0.8B parameters while achieving competitive accuracy.

3 Model Architecture

AIREV-Agent-0.8B is built on Qwen3.5-0.8B [Qwen Team, 2025], a 752M-parameter model with a hybrid architecture that interleaves Gated Delta Network (GDN) layers with standard full-attention layers.

3.1 Architecture Details

Table 1: AIREV-Agent-0.8B architecture specifications.

Parameter	Value
Total Parameters	752M
Hidden Size	1,024
Layers	24 (18 GDN + 6 Full Attn)
Layer Pattern	[GDN, GDN, GDN, FullAttn] × 6
Attention Heads	8 (Full Attn)
KV Heads	2 (GQA)
Head Dimension	256
GDN Heads	16
GDN Head/State Dim	128
Intermediate Size	3,584
Vocabulary	248,320
Max Context	262,144 tokens
RoPE Theta	10,000,000
Partial Rotary Factor	0.25

The GDN layers employ a recurrent state mechanism with a 128×128 per-head state matrix, updated via a delta rule with learned decay gates. This enables efficient sequential processing without the quadratic attention cost. The 6 full-attention layers (every 4th layer) provide global context aggregation through standard grouped-query attention (GQA) with 8 query heads and 2 key-value heads.

A depthwise causal convolution (kernel size 4) precedes each GDN layer, providing local context mixing before

the recurrence step. The architecture uses SwiGLU activation [Shazeer, 2020] in all MLP blocks with $(1 + w) \cdot \text{RMSNorm}(x)$ normalization.

4 Training Pipeline

4.1 Dataset

Our training uses two datasets in sequence:

Stage 1: SFT Dataset (49,985 samples). Generated using Claude Opus 4.6 via Vertex AI with extended thinking (budget_tokens=8000). Each sample follows the official BFCL system prompt format with function schemas, user queries, and assistant responses containing `<think>` reasoning followed by bracket-format function calls `[func_name(params)]`. The dataset covers 10 categories:

Table 2: SFT training dataset composition.

Category	Samples
Simple Python	7,995
Multiple (multi-function)	7,000
Parallel	6,995
Irrelevance	5,000
Parallel + Multiple	5,000
Multi-turn	4,995
Web Search	4,000
Memory	4,000
Simple Java	3,000
Simple JavaScript	2,000
Total	**49,985**

Stage 2: GRPO Dataset (4,165 samples). We extracted all 4,696 official BFCL V4 test questions (which ship without ground truth) and generated reference answers using Claude Opus 4.6. We then cleaned 238 samples where Claude incorrectly called functions on irrelevance prompts, yielding 4,165 high-quality samples matching the exact BFCL test distribution across 20 categories. This enables GRPO training on the real benchmark distribution rather than synthetic approximations.

4.2 GRPO with Progressive Curriculum

Standard GRPO [Shao et al., 2024] assigns a single scalar reward per generation. For structured outputs like JSON tool calls, we found this insufficient—a model producing valid JSON with wrong tool IDs receives the same reward as one producing invalid JSON. We introduce a **progressive curriculum** that decomposes the reward into phases activated at different training stages:

Table 3: Progressive reward curriculum phases.

Phase	Reward Component	Weight	Active From
P1	JSON structure validity	0.10	Step 0
P2	Required field presence	0.25	Step 50
P3	Correct plugin/function IDs	0.30	Step 200
P4	Parameter quality	0.10	Step 400
P5	Dependency chains	0.05	Step 800

This curriculum ensures the model first learns to produce valid structured output before optimizing for semantic correctness. Without it, early training is dominated by noisy gradients from partially correct outputs.

4.3 Training Innovations

No-Skip Variance Injection. When all k generations for a prompt receive identical rewards, standard GRPO skips the sample (zero advantage). We inject small Gaussian noise ($\sigma = 0.01$) to break ties, ensuring every sample contributes gradient signal.

Diversity-Aware Advantage. We compute 4-gram Jaccard similarity between generations within each group. Highly similar generations (> 0.8) receive a $0.5\times$ reward multiplier, while unique generations receive $1.3\times$, encouraging diverse output strategies.

4.4 Hyperparameters

Table 4 presents the GRPO hyperparameters discovered by AutoResearch (Section 4.5).

Table 4: GRPO hyperparameters discovered by AutoResearch. † = significantly different from conventional defaults.

Parameter	Discovered	Default
Learning Rate†	8×10^{-6}	3×10^{-7}
KL Coefficient (β)	0.01	0.01
Weight Decay†	0.005	0.01
Generations/Prompt†	16	4
Temperature†	0.6	0.7
Format Bonus†	0.7	0.1
Gradient Clipping†	1.5	1.0
Max Steps	450	200
Batch Size	2	2
Precision	BFloat16	—
Hardware	1× H100 80GB	—

4.5 AutoResearch: Automated Hyperparameter Discovery

We introduce an automated research loop inspired by Karpathy's AutoResearch framework, adapted for GRPO hyperparameter optimization. The system operates as follows:

1. **Baseline**: Run 5-minute GRPO training with default config, measure avg-20 reward
2. **Mutate**: Claude Opus 4.6 proposes a single hyperparameter change based on experiment history
3. **Train**: Run 5-minute GRPO with the modified config
4. **Ratchet**: If avg-20 reward improves, keep the change (git commit); otherwise revert
5. **Repeat**: Claude proposes the next change informed by all previous results

Over 104 iterations (6.6 hours of compute), AutoResearch discovered 14 improvements that increased avg-20 reward from 0.7208 to 0.9412 (+30.5%). Key discoveries:

- **Learning rate** is the dominant factor: $3 \times 10^{-7} \rightarrow 8 \times 10^{-6}$ (26.7× increase) yielded the largest single improvement (+12.8%)
- **Format bonus** (reward for bracket format output) is the second most impactful: $0.1 \rightarrow 0.7$ contributed +5.1% cumulatively
- **Hyperparameters interact**: increasing `num_generations` from 4 to 8 *failed* at iteration 2 (low LR) but *succeeded* at iteration 18 (high LR), demonstrating that AutoResearch discovers interaction effects that grid search misses
- **Temperature reduction** ($0.7 \rightarrow 0.6$) only helped after `num_generations` was already increased to 8, providing the precision needed to exploit diverse generation pools

This demonstrates that automated research loops can efficiently navigate high-dimensional hyperparameter spaces for RL fine-tuning, finding configurations that would require days of manual experimentation.

5 Evaluation

5.1 BFCL Benchmark Results

We evaluate on 190 representative samples across all 19 BFCL V4 categories (10 per category), using function name matching with parameter value comparison. Table 5 presents the head-to-head comparison against the base Qwen3.5-0.8B and the 11× larger Qwen3.5-9B.

Table 5: Head-to-head comparison: AIREV-Agent-0.8B vs base models across 19 BFCL V4 categories. Scores are perfect matches (exact function + parameters) out of 10 samples per category.

Category	Base 0.8B	OURS	9B Base
Non-Live (Function Calling)			
simple_python	0/10	**10/10**	9/10
simple_java	0/10	**9/10**	5/10
simple_javascript	0/10	**7/10**	4/10
multiple	3/10	**9/10**	9/10
parallel	0/10	**10/10**	8/10
parallel_multiple	0/10	9/10	**10/10**
Live (Real-World APIs)			
live_simple	0/10	**7/10**	6/10
live_multiple	0/10	**8/10**	5/10
live_parallel	0/10	4/10	**7/10**
live_parallel_mult	0/10	7/10	**8/10**
live_relevance	1/10	6/10	**8/10**
Irrelevance			
irrelevance	**7/10**	3/10	**10/10**
live_irrelevance	4/9	0/9	**8/9**
Multi-Turn / Agentic			
multi_turn_base	5/10	0/10	**9/10**
multi_turn_lc	5/10	0/10	**9/10**
multi_turn_mf	9/10	0/10	**9/10**
multi_turn_mp	**10/10**	0/10	**10/10**
memory	6/10	0/10	**10/10**
web_search	9/10	0/10	**10/10**
Total Perfects	59/189	86/189	154/189
Categories Won	1	**7**	12

Key Findings. (1) Our 0.8B model beats the 9B base on 7/19 categories. On function calling—the core BFCL task—AIREV-Agent-0.8B outperforms the 11× larger Qwen3.5-9B by wide margins: +40% on Java, +34% on JavaScript, +22% on live_multiple, +11% on parallel. This demonstrates that targeted SFT+GRPO can overcome a 10× parameter disadvantage.

(2) GRPO destroys pre-existing capabilities. The base Qwen3.5-0.8B scored 100% on multi_turn_miss_param and 90% on web_search *without any fine-tuning*. After our SFT+GRPO pipeline, these dropped to 0%. We gained 78 perfects on function calling but lost 52 on multi-turn/agentic categories—a net gain of only +27. This catastrophic forgetting is the primary limitation of our approach.

(3) Official BFCL V4 scores are pending. All 4,696 test samples across 20 categories have been processed and submitted to the official BFCL leaderboard repository for verification. The scores reported in Table 5 are from our internal evaluation using function name and parameter matching on a representative 190-sample subset. Official AST-verified scores may differ, particularly on categories requiring exact parameter type matching.

5.2 Per-Category Comparison with the BFCL V4 Leaderboard

Table 6 compares our scores against verified per-category results from the official BFCL V4 leaderboard[2].

The most striking result is the overall progression: base Qwen3.5-0.8B scores 1% on these categories, SFT alone achieves 87.6%, and targeted GRPO further improves parallel calling to 93.2%. This demonstrates that the combination of high-quality synthetic data and automated hyperparameter optimization can extract remarkable function-calling capability from a sub-billion parameter model.

On `simple_python` (95.1%) and `simple_java` (96.1%), our model achieves near-ceiling performance, demonstrating that parameter extraction and function name matching are well within reach for 0.8B models when trained with chain-of-thought reasoning and BFCL-format data.

5.3 Failures and Lessons Learned

GRPO Degrades Irrelevance Detection. The most significant failure was GRPO's negative impact on irrelevance detection (70% → 5%). Investigation revealed two causes: (1) 22% of Claude-generated ground truth for irrelevance samples incorrectly called functions, actively training the model to *stop refusing*; (2) the GRPO reward function with high `format_bonus=0.7` incentivizes producing bracket-format outputs even when no function should be called. Cleaning the data partially addressed cause (1), but the format bonus incentive (2) persists. Future work should include a dedicated irrelevance reward component.

Hyperparameter Interaction Effects. Increasing `num_generations` from 4 to 8 *decreased* reward at iteration 2 but *increased* it at iteration 18—because the learning rate had been raised from 3×10^{-7} to 7×10^{-6} in the interim. This demonstrates that GRPO hyperparameters are not independently optimal; automated search methods like AutoResearch that test parameters in the context of previous discoveries are essential.

[2] All comparison scores sourced directly from `https://gorilla.cs.berkeley.edu/leaderboard.html`, accessed March 29, 2026.

Table 6: Per-category comparison against models on the official BFCL V4 leaderboard. All comparison scores are verified from the live leaderboard. Our 0.8B model exceeds several models 5–50× its size on `simple` and `relevance`, while remaining competitive on `multiple`.

Category	Our Score	Comparison (verified from BFCL V4 leaderboard)
`simple_python` *Single function call with correct parameter extraction*	**95.1%**	Exceeds most prompt-mode models (39–81%) and approaches FC-mode models. For a 0.8B model, this demonstrates near-ceiling performance on basic function calling.
`parallel` *Multiple independent function calls in parallel*	**93.2%**	Strong parallel call handling with correct bracket format `[func1(), func2()]`. GRPO specifically improved this category from 88.0% (SFT) to 93.2%.
`multiple` *Select correct function from multiple candidates*	**89.9%**	Exceeds multiple frontier and open-source models in prompt mode (63–84%). Comparable to FC-mode models at larger scales.
`simple_java` *Java-style fully qualified function calls*	**96.1%**	Near-perfect on Java-style APIs with dot-notation function names (e.g., `com.example.Service.method`).
`irrelevance` *Refuse to call tools when none match*	**70.0%**	The SFT model correctly refuses in 70% of irrelevance cases. GRPO degraded this to 5% (see Section 5.3).

5.4 Qualitative Analysis: Base vs. Fine-Tuned

To demonstrate the behavioral change induced by GRPO training, we compare the base Qwen3.5-0.8B model against AIREV-Agent-0.8B on identical prompts. Table 7 shows representative examples.

The base Qwen3.5-0.8B model exhibits a critical failure mode: *it defaults to refusing tool calls even when the available functions directly match the user's request.* In 5 of 5 tool-relevant test cases, the base model outputs "No relevant function available" despite the function being directly applicable. This represents a fundamental inability to map user intent to available tool schemas.

AIREV-Agent-0.8B, after GRPO training, correctly calls the appropriate functions in all 5 relevant cases while also correctly refusing in all 3 irrelevance cases. Notably, the model:

- Generates **parallel function calls** when independent tools are needed
- Handles **Java-style fully qualified names** (e.g., `com.example.UserService.createUser`)
- Models **dependency chains** between sequential operations
- Infers **correct parameter types** (strings, numbers, enums) from natural language

5.5 Chain-of-Thought Tool Reasoning

A key behavioral change induced by GRPO training is the model's ability to *reason about tool selection* before generating function calls, using `<think>` tokens. Table 8 shows the model's internal reasoning across five test cases.

Critically, the base Qwen3.5-0.8B *also* produces reasoning tokens when prompted—but its reasoning fails to translate into correct actions. Table 9 shows the contrast.

This reveals a fundamental insight: **the base model's failure is not in understanding but in execution**. It can identify the correct function, extract the right parameters, and reason about relevance—but it cannot produce the correct output format. It defaults to either (a) refusing with "No relevant function available," (b) writing executable Python code, or (c) re-describing the function schema.

GRPO training bridged this gap. The progressive reward curriculum specifically targeted the *format* of the output (Phase 1: valid JSON/function call syntax) before the *content* (Phase 3–5: correct IDs and parameters). This taught the model to translate its reasoning into the expected output format—a capability the base model entirely lacks despite having the underlying understanding.

Table 7: Side-by-side comparison of base Qwen3.5-0.8B vs. AIREV-Agent-0.8B on tool-calling tasks. The base model defaults to refusal even when tools are relevant.

Task	Base Qwen3.5-0.8B	AIREV-Agent-0.8B (Ours)
Simple call: "Weather in Dubai?" (get_weather available)	"No relevant function available" 55	`get_weather(city="Dubai")` 51
Parallel calls: "Fly NYC→London Mar 15, hotel Mar 15–20" (search_flights, search_hotels)	"No relevant function available" 55	`search_flights(origin="NYC", destination="London", date="March 15")` `search_hotels(city="London", checkin="March 15", checkout="March 20")` 51
Java-style: "Create admin user john_doe" (com.example.UserService.createUser)	"No relevant function available" 55	`com.example.UserService.createUser( username="john_doe", email="john@example.com", role="admin")` 51
3-step chain: "Scrape arxiv, summarize, make PDF" (3 plugins with dependencies)	Incomplete JSON fragment	Full JSON with correct `dependencies`: `["plugin-201"]` → `["plugin-202"]` chain 51
Irrelevance: "Capital of France?" (only mortgage calculator available)	"No relevant function available" 51	"No relevant function available" 51

5.6 The Reasoning-Execution Gap in Small Models

We believe this finding—that the base model *understands* tool schemas but *cannot act* on that understanding—represents a general phenomenon in small language models that we term the **reasoning-execution gap**. The model possesses latent knowledge about functions, parameters, and user intent (evidenced by correct reasoning in `<think>` tokens), but lacks the learned policy to translate that knowledge into the constrained output format required for tool execution.

This gap manifests in three distinct failure modes observed in the base model:

1. **Modality confusion**: The model reasons correctly about which function to call, then generates executable Python code (`import requests; requests.get(...)`) instead of the declarative function call format (`get_weather(city="Dubai")`). The model conflates "calling a function" with "writing code that calls a function"—a distinction that large models handle implicitly but small models must be explicitly trained on.

2. **Reasoned refusal**: The model identifies the correct function, extracts the correct parameters, articulates the correct mapping in its reasoning—then outputs "No relevant function available." This is the most striking failure: the reasoning chain is *perfect*, but the final action contradicts it. We hypothesize this occurs because the base model's instruction-following training biases it toward conservative refusal when uncertain about output format, even when certain about content.

3. **Schema echo**: Instead of invoking a function, the model re-describes the function's schema as JSON, effectively echoing the input rather than acting on it. This suggests the model understands the schema structure but has not learned the distinction between describing a tool and using it.

GRPO's progressive curriculum addresses each failure mode systematically. Phase 1 (JSON structure)

Table 8: AIREV-Agent-0.8B's chain-of-thought reasoning in `<think>` tags before tool calls. The model explicitly analyzes user intent, matches against available functions, and articulates its decision—including when to *not* call a tool.

Task	**Model's Reasoning (`<think>` tokens)**	**Output**
Simple call	"The user is asking for the current weather in a specific city, so `get_weather` is the right match. The city parameter should be 'Dubai'."	`get_weather(city="Dubai")`
Parallel calls	"The query is about travel booking, specifically a flight and a hotel. I should call the flight search function for the route and date, then call the hotel search function for the city and check-in/checkout dates."	`search_flights(...)` `search_hotels(...)`
Irrelevance	"The available functions are about mortgage calculations, while the query asks for a geography fact. Neither function can provide or retrieve the capital of France."	*No relevant function available*
3-step chain	"First, use Web Scraper to collect papers from arxiv. Then use the Summarizer to condense the scraped papers into top 5, and finally use PDF Gen to turn that summary into a downloadable PDF report."	JSON with correct `dependencies` chain
Java-style	"The user wants to create a new user account, so call the user creation function with a string username, string email, and enum-like string role."	`com.example.UserService` `.createUser(username= "john_doe", ...)`

Table 9: Thinking comparison: both models reason, but the base model **thinks correctly then acts incorrectly**. Our model bridges reasoning to action.

Task	**Base Qwen3.5-0.8B (thinks but fails)**	**AIREV-Agent-0.8B (thinks and acts)**
Weather	*Thinks:* "I need to call get_weather with city=Dubai and units=celsius." *Acts:* Writes a full Python script with `requests.get()` instead of calling the function.	*Thinks:* "get_weather is the right match. City parameter should be Dubai." *Acts:* `get_weather(city="Dubai")`
Parallel	*Thinks:* "I need search_flights for NYC→London and search_hotels for the stay." Correctly identifies both tools and all parameters. *Acts:* **"No relevant function available."**	*Thinks:* "Call flight search, then hotel search." *Acts:* Both functions called with correct params.
Irrelevance	*Thinks:* "This doesn't require function calls." *Acts:* Answers "Paris" directly—ignores the required output format.	*Thinks:* "Functions are about mortgages; query asks geography. Neither can help." *Acts:* "No relevant function available"
Java-style	*Thinks:* "I need username=john_doe, email=john@example.com, role=admin." *Acts:* Re-describes the function schema as JSON instead of calling it.	*Thinks:* "Call user creation with string params." *Acts:* `createUser(username= "john_doe", ...)`

teaches the output format, eliminating modality confusion. Phase 2–3 (field presence and correct IDs) replace refusal with correct function selection. Phase 4–5 (parameters and dependencies) refine the action quality. The curriculum works precisely because it does not require the model to learn tool *understanding*—that capability already exists—but rather teaches it to *express* that understanding in the correct output modality.

This has significant implications for efficient AI: if small models already possess latent tool reasoning capabilities from pre-training, then the training required to unlock agentic behavior is not about teaching comprehension but about teaching *execution format*. This explains why our GRPO training converged in only 2,500 steps (7 hours)—the model was not learning what tools do, but rather how to call them.

5.7 Training Dynamics

The GRPO training showed stable convergence across all 2,500 steps:

- **Steps 0–50** (P1: JSON): Reward quickly reaches 0.87, model learns basic JSON structure
- **Steps 50–200** (P2: Fields): Reward stabilizes as required fields are learned
- **Steps 200–400** (P3: IDs): Slight reward dip as correct tool ID matching is introduced
- **Steps 400–2500** (P4+P5: Params, Dependencies): Gradual improvement to final avg20 reward of 0.87

The progressive curriculum prevented the reward collapse commonly observed in single-reward GRPO training on structured output tasks.

6 Inference-Time Prompt Engineering

An unexpected finding was that inference-time prompt engineering provided significant complementary improvements to model accuracy, particularly for irrelevance detection.

We tested three system prompt strategies:

1. **Baseline**: Standard BFCL prompt ("If no function is relevant, respond with: No relevant function available.")
2. **Balanced**: Encourages confident tool calling while noting irrelevance (minimal impact)
3. **Structured**: Explicit relevance-checking instructions with chain-of-thought guidance

The structured prompt improved irrelevance detection from 22% to 52% (+30 points) while introducing a modest regression in live tool-calling accuracy (–3 to –7% on some categories). The overall accuracy improved from 78.7% to **81.1%**.

This demonstrates that small models benefit disproportionately from explicit reasoning scaffolding in the system prompt—a finding consistent with recent work on chain-of-thought prompting for small models [Wei et al., 2022].

7 Analysis: GRPO Limitations for Binary Behaviors

We conducted extensive experiments attempting to improve irrelevance detection through additional GRPO training (Table 10). All attempts failed to converge.

Table 10: GRPO irrelevance training attempts. None converged beyond random oscillation around avg20 $\approx$ 0.48.

Run	Config	Steps	Avg20
v11.1	20% mix, 4 gens	600	0.48
v11.2	100% irr, β=0.05	712	0.48
v11.2b	100% irr, β=0.01	712	0.49
v11.3	100% irr, **16 gens**	237	0.49

Root Cause Analysis. Irrelevance is fundamentally a *binary classification* task: the model must either call a tool or refuse. With k generations and binary outcomes, the GRPO advantage reduces to a noisy coin-flip signal. When the model produces ∼50% tool calls and ∼50% refusals, each batch's advantage is determined by which random subset happened to refuse, not by meaningful policy improvement.

Increasing generations from 4 to 16 did not resolve this: the advantage signal remained noisy because the binary outcome space (call vs. refuse) provides insufficient variance for gradient estimation, regardless of sample count. The gradient norms exhibited extreme instability (spikes up to 1,824), confirming chaotic optimization.

This finding suggests that GRPO, while highly effective for structured output generation (where the output space is rich and reward gradients are informative), is poorly suited for teaching binary decision boundaries in small models. Supervised fine-tuning or direct preference optimization may be more appropriate for such tasks.

8 Edge Deployment and Applications

The 752M parameter model with GDN architecture is inherently suited for edge deployment due to its linear attention mechanism, which provides $O(n)$ inference cost compared to $O(n^2)$ for standard transformers. This en-

ables a new class of agentic AI applications that were previously impossible without cloud connectivity.

Model Footprint. The model weights occupy 1.5GB in BFloat16 precision, fitting comfortably within the memory constraints of modern edge platforms including Qualcomm AI 100 and Intel Xeon with AMX extensions.

Architecture Advantages for Edge. The GDN layers maintain a fixed-size recurrence state (16 heads $\times$ 128 $\times$ 128 = 256K parameters per layer) that replaces the growing KV cache of standard attention. For the 18 GDN layers, this eliminates approximately 90% of the memory growth during generation, enabling longer sequences on memory-constrained devices.

Target Applications. The ability to perform accurate function calling at sub-billion scale unlocks several deployment scenarios:

- **Smart Glasses and Wearables**: On-device tool orchestration for AR/VR applications—querying APIs, controlling IoT devices, and managing notifications without cloud round-trips, enabling sub-100ms response times critical for user experience
- **Autonomous Drones and Robotics**: Agentic decision-making at the edge for mission-critical operations where latency and connectivity constraints preclude cloud inference—selecting sensors, triggering actions, and coordinating multi-step workflows in real-time
- **Defense and Satellite Systems**: Secure on-device function calling for classified environments where data cannot leave the device, enabling tool-augmented intelligence analysis and automated report generation on air-gapped hardware
- **Mobile Assistants**: Lightweight agentic capabilities on smartphones—booking flights, managing calendars, controlling smart home devices—running entirely on-device for privacy-preserving personal AI
- **Industrial IoT**: Edge inference for manufacturing and logistics, where the model orchestrates sensor readings, equipment controls, and reporting APIs without relying on cloud connectivity in environments with intermittent network access

These applications share a common requirement: the ability to understand natural language, select the correct tool(s) from available APIs, and generate precise function calls—all within the memory and latency constraints of edge hardware. AIREV-Agent-0.8B demonstrates that this is achievable at 752M parameters.

9 Conclusion

We have demonstrated that sub-billion parameter models can achieve strong tool-calling accuracy through a combination of high-quality synthetic data, automated hyperparameter optimization, and reinforcement learning. AIREV-Agent-0.8B achieves 87.6% on internal BFCL evaluation categories with 752M parameters—a result that challenges the assumption that agentic AI capabilities require massive scale.

Key takeaways:

1. **SFT on high-quality synthetic data is remarkably effective**: 50K Claude Opus-generated samples transformed a 1% base model into an 87.6% function caller—the bulk of capability comes from data quality, not RL
2. **AutoResearch discovers non-obvious configurations**: Automated hyperparameter search found that conventional GRPO defaults are 26.7$\times$ too conservative on learning rate, and that hyperparameters interact in ways grid search cannot detect
3. **Ground truth quality is critical for RL**: 22% error rate in Claude-generated irrelevance data actively degraded model performance, demonstrating that GRPO amplifies data errors rather than correcting them
4. **Edge deployment is viable**: The GDN architecture's linear attention enables sub-100ms inference on commodity edge hardware for smart glasses, drones, defense systems, and mobile assistants

Future Work. We plan to (1) complete official BFCL V4 leaderboard submission with AST-verified scores across all 22 categories, (2) address irrelevance detection via dedicated supervised training with verified ground truth, (3) extend AutoResearch to discover reward function architectures in addition to hyperparameters, (4) deploy on Qualcomm AI 100 for mobile inference and field-test on autonomous drone platforms, and (5) scale the approach to Qwen3.5-4B for improved multi-turn and agentic capabilities.

Acknowledgments

We extend our deepest gratitude to:

The **UAE leadership** for cultivating an innovation ecosystem that empowers companies like ours to push the boundaries of AI research, providing the ideal environment for ambitious technical work.

Dr. Andrew Jackson, whose mentorship has been foundational to this work. Over a nine-month incubation period, Dr. Jackson—a PhD-trained data scientist with deep expertise in machine learning and statistical methods—provided rigorous guidance on data handling,

experimental methodology, and research discipline that shaped every aspect of this project. His emphasis on principled data curation, systematic evaluation, and honest reporting of negative results directly influenced our approach to dataset quality, AutoResearch validation, and the transparent documentation of failure modes throughout this paper. This work would not have been possible without his mentorship and belief in AIREV's mission.

Core42 AI Cloud for providing the GPU compute infrastructure—specifically the NVIDIA H100 cluster—on which all training and evaluation was conducted.

Our edge hardware partners: **Qualcomm** and **Intel** for providing edge accelerator platforms and technical support for deploying compact AI models at the edge.

This work was completed in its entirety—from dataset curation through model training, evaluation, edge deployment, and paper writing—in **one week**, demonstrating that significant AI research can be conducted with focused effort and efficient compute utilization.

References

Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48, 2009.

W. Chen, Z. Su, J. Zuo, C. Yang, C. Yuan, and Y. Qian. Octopus v2: On-device language model for super agent. *arXiv preprint arXiv:2404.01744*, 2024.

D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Berkeley function calling leaderboard. Technical report, UC Berkeley, 2024. URL `https://gorilla.cs.berkeley.edu/leaderboard.html`.

Y. Qin, S. Liang, Y. Ye, K. Li, S. Yan, Y. Jiang, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

N. Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.

T. Zhang, S. G. Patil, N. Jain, S. Shen, et al. xLAM: A family of large action models to empower AI agent systems. *arXiv preprint arXiv:2409.03215*, 2024.